Códigos de R para Regresión Lineal

Forro — Examen No. 2 / Segundo semestre 2026

Víctor Brizuela · Escuela de Estadística, Universidad de Costa Rica

Contenido

1. Preparar la base **2**

2. Interacción entre un numérico y un factor **2**

2.1. Gráfico por grupos . . . 2
2.2. Modelos y prueba de interacción . . . 2
2.3. Ecuaciones por grupo . . . 3

3. Comparar medias y pendientes **3**

3.1. Una comparación con intervalo de confianza . . . 3
3.2. Varias comparaciones con Bonferroni y una cola . . . 3
3.3. Cambio esperado ante un aumento "importante" de x1 . . . 4
3.4. Igualar pendientes o unir categorías (modelo pequeño) . . . 4

4. Selección de variables **5**

4.1. R^2 ajustado con leaps (todos los subconjuntos, sin interacciones) . . . 5
4.2. Paso a paso con drop1 (hacia atrás) . . . 6
4.3. Paso a paso con add1 (hacia adelante) . . . 6
4.4. Automático con step . . . 6
4.5. Ver el AIC, la verosimilitud y la devianza . . . 7
4.6. R^2, R^2 ajustado y R^2 parcial . . . 7
4.7. Cp de Mallows (sesgo) . . . 7
4.8. PRESS y P^2 . . . 7

5. Valores extremos e influencia **8**

5.1. Valor extremo en Y (residuos estudentizados externos) . . . 8
5.2. Influencia sobre su propio ajustado: DFFITS . . . 8
5.3. Influencia sobre todos los ajustados: distancia de Cook . . . 8
5.4. Influencia sobre cada coeficiente: DFBETAS . . . 9
5.5. Comentar la influencia sobre un coeficiente (reajustar y comparar) . . . 10
5.6. Residuo externo sin usar el leverage . . . 10

6. Otros cálculos a mano **10**

6.1. Prueba F entre modelos anidados . . . 10
6.2. Log-verosimilitud y AIC a mano . . . 11
6.3. AIC a mano desde las SCRes (por ejemplo, desde una salida de drop1) . . . 11

1. Preparar la base

```
load("datos.Rdata")                       # carga el objeto del archivo (casi siempre se llama base)
base = as.data.frame(base)                # solo si viene como matriz: permite usar base$variable
library(car); library(lmtest); library(lattice); library(leaps)

base$f = factor(base$f)                   # convierte el código numérico (1, 2, 3...) en factor
levels(base$f) = c("A", "B", "C")         # etiqueta los niveles en el orden de los códigos
contrasts(base$f)                         # la fila de ceros es la categoría de referencia
base$f = relevel(base$f, ref = "C")       # cambia la referencia si la comparación lo pide
table(base$f)                             # cuántos casos hay en cada categoría
nrow(base)                                # n, total de unidades de observación
base$logy = log(base$y)                   # respuesta en logaritmo, cuando el ejercicio lo pide
```

Notas

- `contrasts` se corre siempre antes de escribir el modelo: dice qué indicadoras crea R y con qué nombre aparecen en el `summary` (por ejemplo `fB`, `fC`).
- `table` sirve para comentar qué tan confiable es la pendiente de cada grupo: con pocos casos, poco confiable.
- Si se hace una sub-base con algunas categorías, se agrega `base2$f = droplevels(base2$f)` para eliminar los niveles vacíos.

2. Interacción entre un numérico y un factor

2.1 Gráfico por grupos

```
xyplot(y ~ x1, groups = f, type = c("p", "r"), auto.key = TRUE, data = base)
```

Notas

- `groups = f` pinta cada categoría de un color; `type = c("p","r")` dibuja los puntos (`p`) y la recta de regresión de cada grupo (`r`); `auto.key = TRUE` agrega la leyenda.
- Lectura: rectas claramente no paralelas sugieren interacción. El gráfico solo sugiere; la conclusión sale de la prueba.

2.2 Modelos y prueba de interacción

```
mod1 = lm(y ~ x1*f, data = base)          # con interacción: x1 + f + x1:f
mod0 = lm(y ~ x1 + f, data = base)        # sin interacción: rectas paralelas
anova(mod0, mod1)                         # F de H0: todos los coeficientes de x1:f son 0
summary(mod1)                             # si f tiene 2 categorías, la t de x1:f da la misma prueba
```

Notas

- `x1*f` incluye ambos efectos principales y su interacción; `x1:f` es solo el producto.
- En `anova(mod0, mod1)` el modelo pequeño va primero. Los gl del numerador (columna `Df`) son $k-1$,

la cantidad de coeficientes de interacción.

- Con $k > 2$ categorías no sirve el `summary`: hay que usar la F.

2.3 Ecuaciones por grupo

```
b = mod1$coef                          # orden: (Intercept), x1, fB, fC, x1:fB, x1:fC
round(c(b[1], b[2]), 4)                # referencia A: intercepto y pendiente
round(c(b[1] + b[3], b[2] + b[5]), 4)  # grupo B: se suman su indicadora y su interacción
round(c(b[1] + b[4], b[2] + b[6]), 4)  # grupo C
```

Notas

- Antes de sumar posiciones conviene correr `names(b)` para confirmar el orden de los coeficientes.
- En el modelo sin interacción todas las pendientes son `b[2]` y solo cambia el intercepto.

3. Comparar medias y pendientes

3.1 Una comparación con intervalo de confianza

```
b = mod$coef                           # coeficientes estimados
xh = c(0, 0, -1, 1, -40, 40)           # un peso por coeficiente, en el orden de b
yh = t(xh) %*% b                        # estimación de la diferencia: xh'b
ee = sqrt(t(xh) %*% vcov(mod) %*% xh)   # error estándar: raíz de xh' V xh
gl = mod$df.residual                   # n - p
t = qt(0.975, gl)                      # valor t para 95% bilateral
c(yh - t*ee, yh + t*ee)                # intervalo de confianza
```

Notas

- El vector `xh` sale de escribir la diferencia en coeficientes. Con `mod = lm(y ~ x1*f)` y orden (Int, x1, fB, fC, x1:fB, x1:fC), C menos B en $x_1 = 40$ es $(\lambda_C - \lambda_B) + (\delta_C - \delta_B) \cdot 40$, que da $(0, 0, -1, 1, -40, 40)$.
- Pendiente de un grupo $(\beta_1 + \delta_B)$: `xh = c(0, 1, 0, 0, 1, 0)`.
- Si el intervalo contiene el 0, no se detecta diferencia en ese valor de x_1.
- Para un **cambio** en la media ("cuánto aumenta") el intercepto se cancela; para el **valor** de la media en un x dado, sí entra `b[1]`.

3.2 Varias comparaciones con Bonferroni y una cola

```
c1 = c(0, 0, 0, -1, 0, -t1, 0)         # una comparación por vector (t1 = valor fijo de x1)
c2 = c(0, 0, 0, 0, -1, 0, -t1)
c3 = c(0, 0, 0, -1, 1, -t1, t1)
xh = cbind(c1, c2, c3)                 # cada columna es una comparación
yh = t(xh) %*% b                        # las tres diferencias estimadas
ee = sqrt(diag(t(xh) %*% vcov(mod) %*% xh))   # error estándar de cada una (la diagonal)
q = abs(yh)/ee                         # estadístico t de cada comparación
prob = pt(q, mod$df.residual, lower.tail = F) # probabilidad de una cola
prob < 0.05/3                          # TRUE = hay diferencia (Bonferroni con m = 3)
```

```
qt = qt(1 - 0.05/3, mod$df.residual)    # valor t para la cota de una cola
yh - qt*ee                              # cota inferior de cada diferencia
```

Notas

- `diag(...)` toma solo las varianzas de cada comparación. Los vectores de ejemplo son para un modelo de 7 coeficientes con orden (Int, x1, g2, f2, f3, x1:f2, x1:f3), donde g es otro factor de 2 niveles. Siempre se arman según el orden de `names(b)`.
- Para una cola se compara con $0.05/m$; para intervalos bilaterales se usa `qt(1 - 0.05/(2*m), gl)`.
- Se pone primero la categoría con media mayor para que la diferencia salga positiva.

3.3 Cambio esperado ante un aumento "importante" de x1

```
s = sd(base$x1)             # una desviación estándar de x1 como cambio con sentido
b[2]*s                      # cambio en la media del grupo de referencia
(b[2] + b[5])*s             # cambio en la media del grupo B (pendiente con su interacción)
```

Notas

Con respuesta en logaritmo, el cambio multiplicativo es `exp(b[2]*s)`.

3.4 Igualar pendientes o unir categorías (modelo pequeño)

Modelo grande con k categorías (referencia $= 1$): $\mu = \beta_0 + \beta_1 X + \sum_j \alpha_j D_j + \sum_j \gamma_j X D_j$. Recta de la referencia: $\beta_0 + \beta_1 X$; de la categoría j: $(\beta_0 + \alpha_j) + (\beta_1 + \gamma_j)X$.

Regla general

El modelo pequeño es el modelo grande con H_0 impuesta.

1. Escribir la recta de cada categoría.
2. Igualar lo que pide la pregunta: solo pendientes (1 restricción) o intercepto **y** pendiente para unir categorías (2 restricciones).
3. Sustituir H_0 en el modelo grande: los términos con el mismo coeficiente se juntan en una binaria nueva ($D_{ij} = D_i + D_j$) o desaparecen.
4. `anova(pequeño, grande)`: los gl del numerador son el número de restricciones.

Caso 1: misma pendiente para dos categorías i y j (no referencia). $H_0 : \gamma_i = \gamma_j$ (1 gl). El intercepto de cada una queda libre.

```
mod.g <- lm(y ~ x * f, data = base)                  # grande
base$conjunto <- ifelse(base$f %in% c("cat_i","cat_j"), 1, 0)
mod.p <- lm(y ~ x + f + x:conjunto, data = base)     # una pendiente comun para i y j
anova(mod.p, mod.g)                                  # H0: gamma_i = gamma_j

# Alternativa sin modelo pequeno: contraste (1 gl, F = t^2)
b  <- mod.g$coef
cv <- c(0, 0, 0, 0, 1, -1)              # posiciones de gamma_i y gamma_j (k = 3)
L  <- t(cv) %*% b
ee <- sqrt(t(cv) %*% vcov(mod.g) %*% cv)
2 * pt(-abs(L/ee), mod.g$df.residual)   # p de dos colas
```

Caso 2: unir dos categorías (misma recta completa). Si una es la referencia: $H_0 : \alpha_j = \gamma_j = 0$. Si ninguna lo es: $H_0 : \alpha_i = \alpha_j$ y $\gamma_i = \gamma_j$. Siempre 2 gl.

```
mod.g <- lm(y ~ x * f, data = base)                  # grande: k rectas
base$f2 <- as.character(base$f)
base$f2[base$f2 %in% c("cat_i","cat_j")] <- "cat_ij" # misma etiqueta = misma categoria
base$f2 <- factor(base$f2)
mod.p <- lm(y ~ x * f2, data = base)                 # pequeno: k-1 rectas
anova(mod.p, mod.g)                                  # p > alfa: se pueden unir
```

Cuidado

- Para unir categorías **no basta** con que las pendientes sean iguales: dos rectas paralelas pueden estar a distinta altura.
- Usar el modelo sin interacción como pequeño impone que **todas** las pendientes sean iguales, que es más de lo que pide la pregunta.
- Sin modelo pequeño: mirar en el `summary` los dos coeficientes (α_j y γ_j) con Bonferroni, comparando cada p contra $\alpha/2$. Solo sirve para **rechazar** (no unir); si nada sale significativo, no se puede concluir que se puedan unir.

4. Selección de variables

4.1 R² ajustado con leaps (todos los subconjuntos, sin interacciones)

```
x = with(base, cbind(x1, x2, x3, x4, f))      # matriz con todos los predictores candidatos
y = base$y                                    # respuesta
sel = leaps(x, y, method = "adjr2")           # ajusta todos los modelos posibles y guarda su R^2
    ajustado
k = nrow(sel$which)                           # cuántos modelos se probaron
mat = cbind(1:k, sel$which, round(sel$adjr2, 3))   # número de modelo, variables (1 = está) y R^2 aj
max(sel$adjr2)                                # el máximo R^2 ajustado
mat[sel$adjr2 >= max(sel$adjr2) - 0.02, ]     # modelos a no más de 0.02 (2%) del máximo
mod4 = lm(y ~ x1 + x2 + f, data = base)       # ajustar el modelo elegido (con f como factor)
round(mod4$coef, 4)                           # coeficientes para escribir la ecuación
```

Notas

- `with(base, ...)` permite escribir los nombres de las variables sin `base$`. `leaps` necesita una matriz numérica, por eso se arma con `cbind`.
- Las columnas de `sel$which` siguen el orden del `cbind` (1 = x1, 2 = x2, ...). La primera columna de `mat` sin nombre indica cuántas variables tiene cada modelo.
- `mat[49, ]` solo muestra la fila de la tabla; para usar ese modelo hay que ajustarlo con `lm`.
- Criterio: entre los modelos dentro del margen se elige el que tenga menos variables (parsimonia).

4.2 Paso a paso con drop1 (hacia atrás)

```
mod = lm(y ~ ., data = base)          # modelo con todas las columnas de la base como predictores
drop1(mod, test = "F")                # F, probabilidad y AIC de quitar cada término
drop1(mod)                            # solo AIC (es el criterio por defecto)
mod = update(mod, . ~ . - x3)         # quitar x3 y volver a correr drop1
```

4.3 Paso a paso con add1 (hacia adelante)

```
mod0 = lm(y ~ 1, data = base)                    # solo intercepto: predice la media
add1(mod0, scope = ~ x1 + x2 + x3 + f)           # AIC al agregar cada candidata
mod0 = update(mod0, . ~ . + x1)                  # agregar la que deja el AIC más bajo
```

	Hacia atrás (`drop1`)	**Hacia adelante (`add1`)**
Se parte de	Modelo completo	Modelo vacío `y ~ 1`
En cada paso	Se quita el término cuya fila tiene el AIC más bajo, si es menor que `<none>` (con F: el de mayor p)	Se agrega la variable que deja el AIC más bajo, si es menor que `<none>`
Se detiene cuando	Quitar cualquiera sube el AIC (con F: todas las $p < \alpha$)	Agregar cualquiera sube el AIC
Interacciones	Primero se quitan las interacciones; `drop1` no ofrece sus variables	Entran después de sus variables; deben estar en el `scope`

4.4 Automático con step

```
mod_atras = step(mod)                            # hacia atrás con AIC desde el modelo completo
mod_bic = step(mod, k = log(nrow(base)))         # hacia atrás con BIC (multa log(n) por parámetro)
mod_adel = step(mod0, direction = "forward",
                scope = ~ x1 + x2 + x3 + f)      # hacia adelante con AIC desde el modelo vacío
summary(mod_atras)                               # modelo resultante
```

Notas

- `step` solo considera las interacciones que estén en el modelo inicial (hacia atrás) o en el `scope` (hacia adelante).
- Imprime cada paso: esas tablas justifican cada eliminación o entrada. El BIC multa más y suele dar un modelo más pequeño.

4.5 Ver el AIC, la verosimilitud y la devianza

```
AIC(mod)               # -2 logL + 2k, con k = coeficientes + 1
extractAIC(mod)        # n log(SCRes/n) + 2p: la versión de drop1 y step
logLik(mod)            # log-verosimilitud
deviance(mod)          # en un lm es la SCRes
AIC(mod1); AIC(mod2)   # comparar dos modelos siempre con la misma función
```

Criterio	Se prefiere el modelo con
AIC, BIC, PRESS	Menor
P^2, R^2 ajustado	Mayor
C_p de Mallows	C_p cercano o menor que p

4.6 R², R² ajustado y R² parcial

```
summary(mod)$r.squared                                    # R^2
summary(mod)$adj.r.squared                                # R^2 ajustado
(deviance(mod_sin) - deviance(mod_con))/deviance(mod_sin)  # R^2 parcial de la variable que difiere
```

4.7 Cp de Mallows (sesgo)

```
k  = nrow(anova(modc))           # última fila del anova del modelo completo (Residuals)
k1 = nrow(anova(mod1))           # última fila del anova del modelo evaluado
cme = anova(modc)[k, 3]          # CMRes del modelo completo (columna Mean Sq)
scr = anova(mod1)[k1, 2]         # SCRes del modelo evaluado (columna Sum Sq)
p = length(mod1$coef)            # coeficientes del modelo evaluado
cp = scr/cme - (nrow(base) - 2*p)
p; cp                            # Cp cercano o menor que p: no hay sesgo
```

4.8 PRESS y P²

```
r = mod$res                      # residuos ordinarios
h = hatvalues(mod)               # leverages
d = r/(1 - h)                    # residuos de predicción (sin el caso i)
PRESS = sum(d^2)                 # suma de cuadrados de predicción
n = nrow(base)
SCT = var(base$y)*(n - 1)        # suma de cuadrados total
SCTi = (n/(n - 1))^2*SCT         # SCT prediciendo cada y con la media de los demás
P2 = 1 - PRESS/SCTi              # se prefiere el modelo con mayor P^2 (o menor PRESS)
```

5. Valores extremos e influencia

Medida	Qué detecta	Límite
`rstudent`	Valor extremo en Y	$t_{1-\alpha/(2n),\, n-p-1}$ (Bonferroni)
`dffits`	Influencia sobre su propio ajustado	1, o $2\sqrt{(p+1)/(n-p-1)}$
`cooks.distance`	Influencia sobre todos los ajustados	Percentiles 0.10, 0.20, 0.50 de $F(p, n-p)$
`dfbetas`	Influencia sobre cada coeficiente	1, o $2/\sqrt{n}$

5.1 Valor extremo en Y (residuos estudentizados externos)

```
n = nrow(base); p = length(mod$coef)        # n observaciones, p coeficientes (con intercepto)
t = rstudent(mod)                           # residuos estudentizados externamente
tc = qt(1 - 0.05/(2*n), n - p - 1)          # valor crítico de Bonferroni (bilateral, n pruebas)
plot(abs(t)); abline(h = tc)                # los puntos sobre la línea son extremos
subset(cbind(base, t), abs(t) > tc)         # mostrar los casos extremos con sus datos
```

5.2 Influencia sobre su propio ajustado: DFFITS

```
df = dffits(mod)                            # un valor por caso
lim = 2*sqrt((p + 1)/(n - p - 1))           # límite del laboratorio para muestras grandes
plot(abs(df), ylab = "DFFITS")
abline(h = c(1, lim), lty = c(1, 2))        # línea continua = 1, punteada = límite de muestras grandes
subset(cbind(base, df), abs(df) > lim)      # casos que superan el límite, con sus datos
```

Límites

$|\text{DFFITS}| > 1$ en muestras pequeñas o medianas. En muestras grandes, $2\sqrt{p/n}$ (presentación) o $2\sqrt{(p+1)/(n-p-1)}$ (laboratorio). Siempre se dice cuál se usó.

Interpretación

Un caso que supera el límite tiene influencia sobre **su propia estimación**: si se eliminara, su valor ajustado cambiaría de forma importante. Se nombran los casos y se señalan los que sobresalen más, porque con n grande es esperable que algunos pasen el límite por poco. Ejemplo: "Hay 4 hospitales que tienen influencia sobre su estimación individual; de esos, los que tienen más influencia son el 47 y el 112." Si ninguno supera el límite: "Ningún caso tiene influencia importante sobre su propio valor ajustado."

5.3 Influencia sobre todos los ajustados: distancia de Cook

```
cook = cooks.distance(mod)
lim = qf(c(0.1, 0.2, 0.5), p, n - p)        # percentiles 10, 20 y 50 de F(p, n-p)
plot(cook, ylab = "Distancia de Cook")
abline(h = lim, col = c(1, 4, 2))           # negra = 0.10, azul = 0.20, roja = 0.50
which(cook > lim[2])                        # casos con influencia media o fuerte
cook[47]                                    # distancia de Cook de un caso
pf(cook[47], p, n - p)                      # percentil exacto de ese caso
max(cook); pf(max(cook), p, n - p)          # el caso más influyente: si su percentil < 0.10 (o 0.20),
```

```
    ninguno influye
plot(cook, ylim = c(0, max(cook, lim)))    # si no se ven las líneas: amplía el eje Y hasta la más alta
abline(h = lim, col = c(1, 4, 2))
```

Límites

Debajo de la línea **azul** (percentil < 0.20) $\Rightarrow$ **poca influencia**
Entre la **azul** y la **roja** (0.20 a 0.50) $\Rightarrow$ **influencia media**
Sobre la línea **roja** (percentil ≥ 0.50) $\Rightarrow$ **influencia fuerte**

Interpretación

La distancia de Cook mide la influencia sobre **todos los valores ajustados a la vez**, es decir, sobre el ajuste en general. Se nombra el caso y su nivel. Ejemplo: "El caso 12 tiene influencia media sobre las estimaciones en general ($D = 0.40$, percentil 0.24 de la F)." Si nadie llega a la primera línea: "Ninguna distancia de Cook supera el percentil 0.10 de la F, así que no hay casos que influyan sobre el ajuste en general."

5.4 Influencia sobre cada coeficiente: DFBETAS

```
dfb = dfbetas(mod)                          # matriz: una fila por caso, una columna por coeficiente
matplot(abs(dfb[, -1]), pch = 18)           # todos los casos y coeficientes en un gráfico (sin
    intercepto)
abline(h = c(1, 2/sqrt(n)), lty = c(1, 2)) # línea continua = 1, punteada = 2/raíz(n)
which(apply(abs(dfb[, -1]), 1, max) > 1)    # casos con algún |DFBETAS| mayor que el límite
i = 47                                      # el caso que sobresale (o el que señaló Cook)
round(dfb[i, ], 2)                          # DFBETAS de ese caso en cada coeficiente
abs(dfb[i, ]) > 2/sqrt(n)                   # TRUE = ese coeficiente se ve afectado
```

Notas

- `dfb[, -1]` quita la columna del intercepto; `abs` toma el tamaño del cambio sin importar el signo.
- `matplot` grafica cada columna con un color: en el eje X están los casos y cada color es un coeficiente.
- `apply(..., 1, max)` toma, para cada caso (fila), su |DFBETAS| más grande.

Límites

|DFBETAS| > 1 en muestras pequeñas o medianas; $2/\sqrt{n}$ en muestras grandes.

Interpretación

DFBETAS es el cambio en cada coeficiente al quitar el caso, medido en errores estándar. En el gráfico se busca un caso cuyos puntos se separen claramente de la franja del resto; el color del punto más alto dice qué coeficiente afecta más. Ejemplo: "El caso 12 tiene un impacto importante sobre los coeficientes de alimento y partos (|DFBETAS| de 0.87 y 0.93, muy por encima del resto)." Si todos quedan en la franja baja: "Ninguno de los casos genera un cambio importante en los coeficientes, ya que el indicador es menor a 1 en todos los casos."

5.5 Comentar la influencia sobre un coeficiente (reajustar y comparar)

```
plot(abs(dfb[, 2]), pch = 18, main = "x1")   # DFBETAS de un solo coeficiente (columna 2 = x1)
abline(h = 2/sqrt(n))                        # ¿el caso i sobresale sobre el límite?
base1 = base[-i, ]                           # base sin el caso i (o base[cook < 0.5, ])
mod_sin = lm(y ~ x1 + x2 + f, data = base1)  # mismo modelo sin el caso
summary(mod)$coef                            # con el caso
summary(mod_sin)$coef                        # sin el caso: comparar Estimate, Std. Error y Pr(>|t|)
```

Se comparan tres cosas del coeficiente que preguntan:

1. **El coeficiente:** si sube o baja al quitar el caso, y cuánto (en valor y en porcentaje).
2. **El error estándar:** si baja, el caso estaba agregando incertidumbre; si sube, el caso daba precisión (por ejemplo, un x alejado que anclaba la recta).
3. **La significancia:** si la conclusión cambia (pasa de significativo a no significativo o al revés).

Plantilla de interpretación

"Al eliminar el caso [i], el coeficiente de [x] pasa de [. . .] a [. . .] (sube/baja [. . .] %) y su error estándar pasa de [. . .] a [. . .]. El caso estaba [inflando/reduciendo] el efecto estimado de [x] y [aumentando/reduciendo] su incertidumbre. La conclusión sobre [x] [se mantiene/cambia]: [sigue siendo/deja de ser] significativo."

Ejemplo (becas): "Al eliminar el caso 47, el coeficiente de GRADO baja de 2808.20 a 2696.85 (alrededor de 4 %) y su error estándar baja de 146.14 a 134.37. El caso estaba inflando el efecto del grado sobre el monto y su incertidumbre, pero GRADO sigue siendo claramente significativo en ambos modelos."

Cuidado

Si se elimina el caso, todo lo que sigue en el ejercicio se hace con la base nueva (`base1`), y hay que decirlo.

5.6 Residuo externo sin usar el leverage

```
mod10 = lm(y ~ x1, data = base[-10, ])              # ajustar sin el caso 10
y10 = predict(mod10, data.frame(x1 = base$x1[10]))  # predecir el caso 10 con ese modelo
base$y[10] - y10                                    # residuo externo d_10
mod$res[10]/(1 - hatvalues(mod)[10])                # verificación: d = r/(1 - h)
```

6. Otros cálculos a mano

6.1 Prueba F entre modelos anidados

```
anova(mod_peq, mod_gde)                          # automático: el pequeño va primero
q = length(mod_gde$coef) - length(mod_peq$coef)  # cuántos coeficientes se anulan
num = (deviance(mod_peq) - deviance(mod_gde))/q  # aumento de SCRes por coeficiente quitado
den = deviance(mod_gde)/mod_gde$df.residual      # CMRes del modelo grande
F = num/den
1 - pf(F, q, mod_gde$df.residual)                # probabilidad asociada
```

Notas

- Si solo te dan SCRes y CMRes (sin datos), se usa la misma fórmula con esos números.
- Desde una salida de `drop1` con RSS: $\text{SCRes}_{\text{peq}}$ es la fila del término, $\text{SCRes}_{\text{gde}}$ es `<none>` y q es su `Df`.

6.2 Log-verosimilitud y AIC a mano

```
n = nrow(base)
b = mod$coef
mu = b[1] + b[2]*base$x1 + b[3]*base$x2          # media condicional estimada de cada caso
res = base$y - mu                                # residuos (y = nombre real de la respuesta)
sig2 = sum(res^2)/n                              # sigma^2 de máxima verosimilitud (se divide entre n)
logv = log(prod(dnorm(base$y, mu, sqrt(sig2))))  # log del producto de densidades normales
logv; logLik(mod)                                # deben coincidir
aic = -2*logv + 2*(length(b) + 1)                # AIC: k = coeficientes + 1 (cuenta sigma^2)
```

Cuidado

- `logLik` solo recibe un modelo ajustado, no un número.
- `dnorm` recibe la desviación estándar, no la varianza: por eso va `sqrt(sig2)`.
- Si `prod` da 0 (muchos datos), se usa `sum(log(dnorm(...)))`, que es lo mismo sin redondear a cero.
- Si el ejercicio da las log-verosimilitudes, solo se aplica `-2*logv + 2*k`.

6.3 AIC a mano desde las SCRes (por ejemplo, desde una salida de drop1)

```
n*log(SCRes/n) + 2*p        # misma escala que drop1 y extractAIC; p = coeficientes de ESE modelo
```

Por qué funciona

Sustituyendo la verosimilitud normal con $\hat{\sigma}^2 = \text{SCRes}/n$:

$$-2\log L = n\log\left(\frac{\text{SCRes}}{n}\right) + \underbrace{n\log(2\pi) + n}_{\text{igual en todos los modelos}}$$

La constante se cancela al comparar modelos con los mismos datos. En cada fila de `drop1`, p es el número de coeficientes del modelo *sin* ese término: el del modelo completo menos su `Df`.